UNIT 3

NOSQL & MONGODB ARCHITECTURE

✦ What is NoSQL?

NoSQL means “Not Only SQL”. It is a type of database that:

- Does not use tables and rows
- Stores data in flexible formats
- Works well with large-scale applications

Types of NoSQL Databases

Type Example Data Format
Document MongoDB JSON-like
Key-Value Redis Key–Value
Column Cassandra Columns
Graph Neo4j Nodes & Edges

What is MongoDB?

MongoDB is a Document-Oriented NoSQL Database.

- Stores data as documents
- Uses BSON (Binary JSON)
- Schema-less (Flexible structure)

MongoDB Architecture

Client → Application → MongoDB Server → Database → Collection → Document

Component Description
Database Container for collections
Collection Group of documents

Document JSON-like record
Server Stores databases

Example Document

```
{
"name": "Seema",
"age": 40,
"subject": "Computer Science"
}
```

CRUD Operations in MongoDB

CRUD stands for:

Operation	Meaning
C	Create
R	Read
U	Update
D	Delete

Create (Insert Data)

Insert One

```
db.students.insertOne({
name: "Rahul",
age: 20,
course: "BSc IT"
})
```

Insert Many

```
db.students.insertMany([
{ name: "Anu", age: 21 },
{ name: "John", age: 22 }
])
```

Read (Find Data)

Find All
db.students.find()
Find Specific
db.students.find({ age: 21 })

Update (Modify Data)

Update One

```
db.students.updateOne(
{ name: "Rahul" },
{ $set: { age: 22 } }
)
```

Update Many

```
db.students.updateMany(
{ course: "BSc IT" },
{ $set: { department: "IT" } }
)
```

Delete (Remove Data)

Delete One
db.students.deleteOne({ name: "John" })

- Delete Many
  db.students.deleteMany({ age: { $lt: 20 } })

Schema, Models & Validation with Mongoose

What is Mongoose?

Mongoose is a Node.js library used to connect MongoDB with applications.

It helps to:
- Create schemas
- Validate data
- Perform queries

Installing Mongoose

```
npm install mongoose
```

Connecting to MongoDB

```
const mongoose = require("mongoose");

mongoose.connect("mongodb://localhost:27017/collegeDB")
.then(() => console.log("Connected"))
.catch(err => console.log(err));
```

Creating Schema

A Schema defines structure of documents.

```
const studentSchema = new mongoose.Schema({
name: String,
age: Number,
course: String
});
```

Adding Validation

```
const studentSchema = new mongoose.Schema({
name: {
type: String,
required: true
},
age: {
type: Number,
min: 18,
max: 30
},
course: {
type: String,
enum: ["BSc IT", "BCA", "MSc"]
}
});
```

Creating Model

A Model represents a collection.

```
const Student = mongoose.model("Student", studentSchema);
```

Inserting Using Mongoose

```
const stu1 = new Student({
name: "Anu",
age: 21,
course: "BSc IT"
});
```

stu1.save();

Indexing & Relationships

Indexing

Indexes improve search speed.
Create Index
db.students.createIndex({ name: 1 })

1 = Ascending
-1 = Descending

Example
Without index → Slow search
With index → Fast search

Relationships in MongoDB

MongoDB does not use joins like SQL. It uses:

1. Embedded Documents

```
{
name: "Rahul",
address: {
city: "Mumbai",
pincode: 400001
}
}
```

---

2. Referenced Documents

```
{
name: "Rahul",
courseId: ObjectId("12345")
}
```

Mongoose Reference Example

```
const courseSchema = new mongoose.Schema({
name: String
});
const studentSchema = new mongoose.Schema({
name: String,
course: {
type: mongoose.Schema.Types.ObjectId,
ref: "Course"
}
});
```

Using Populate

```
Student.find().populate("course");
```

✦ **Aggregation Framework & Pipelines**

What is Aggregation?

Aggregation is used for data analysis and processing.

It works like a pipeline (step-by-step process).

🔷 Common Stages

Stage Purpose
$match Filter
$group Group
$sort Sort
$project Select fields
$sum Total
$avg Average

Example Collection

```
{
name: "Anu",
course: "IT",
marks: 80
}
```

🔷 Example Pipeline

Find Average Marks per Course

```
db.students.aggregate([
{ $group: { _id: "$course", avgMarks: { $avg: "$marks" } } }
])
```

Filter + Sort

```
db.students.aggregate([
{ $match: { marks: { $gt: 70 } } },
{ $sort: { marks: -1 } }
])
```

———

MongoDB Atlas Setup

🔹 What is MongoDB Atlas?

MongoDB Atlas is a cloud database service.

• No local installation needed
• Free tier available
• Secure and scalable

———

🔷

Steps to Setup

1. Go to: mongodb.com/atlas
2. Create account
3. Create Cluster
4. Select Free Tier
5. Create Username & Password
6. Allow IP (0.0.0.0/0)
7. Get Connection String

———

🔹 Connection Example

```
mongoose.connect(
"mongodb+srv://username:password@cluster0.mongodb.net/collegeDB"
);
```